Neural Ordinary Differential Equations

Ricky T. Q. Chen*, Yulia Rubanova*, Jesse Bettencourt*, David Duvenaud
University of Toronto, Vector Institute
{rtqichen, rubanova, jessebett, duvenaud}@cs.toronto.edu

2026.4.2 张雨阳

node的优势

1. Powerful representation: 微分方程可以用数值法求解，因此对于任何连续函数都有良好的逼近能力

2. Memory efficiency: 向前计算过程不需要构建整张计算图，因此训练上节约内存

3. Simplicity: 不需要考虑复杂的调参和网络设计，形式简洁

4. Abstraction: 让网络不需要考虑每层需要做什么，只需要考虑怎么计算结果

主要内容

1.向前计算过程

2.向后计算过程

3.优势与应用

主要内容

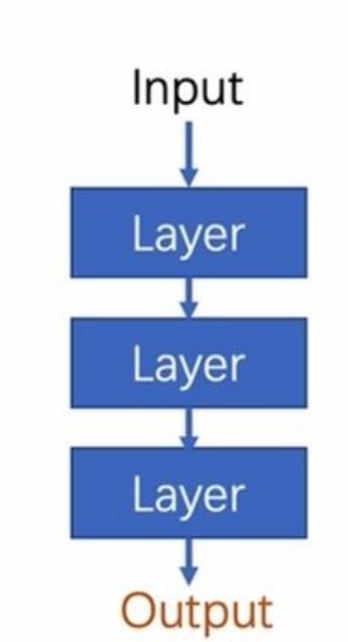
1.向前计算过程

2.向后计算过程

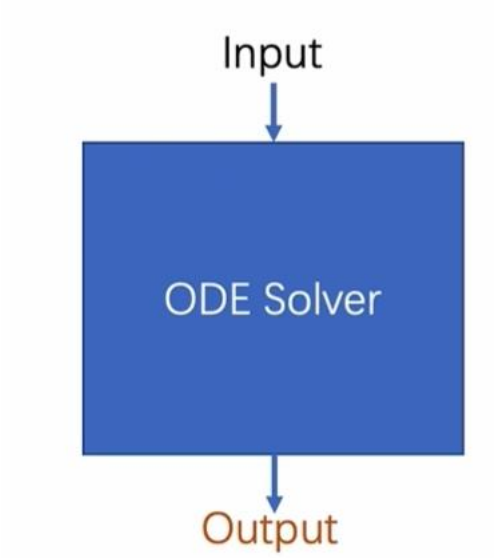
3.优势与应用

对比不同视角下的神经网络

普通神经网络



使用ode solver的神经网络



ode solver（微分方程求解器）
内部使用极其复杂的数值积分算法
用来求解微分方程的解

ODE solver

$$\frac{dy}{dx} = f(x, y), \quad y_0 = y(x_0)$$

已知 (x_0, y_0)，能否计算出 $y(x_0 + h)$？

$$\int_{y_0}^{y_1} dy = \int_{x_0}^{x_1} f(x, y) dx$$

$$y_1 = y_0 + \int_{x_0}^{x_1} f(x, y) dx$$

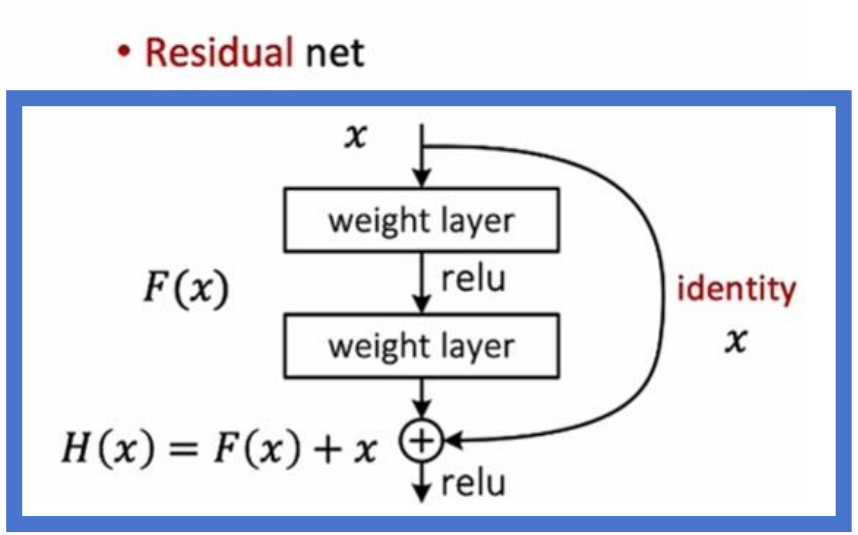
如果 $f(x, y)$ 很复杂，不好求精确表达式。

IPV问题(initial value problems)

给定初始条件的常微分方程求解的问题

残差网络

ode网络



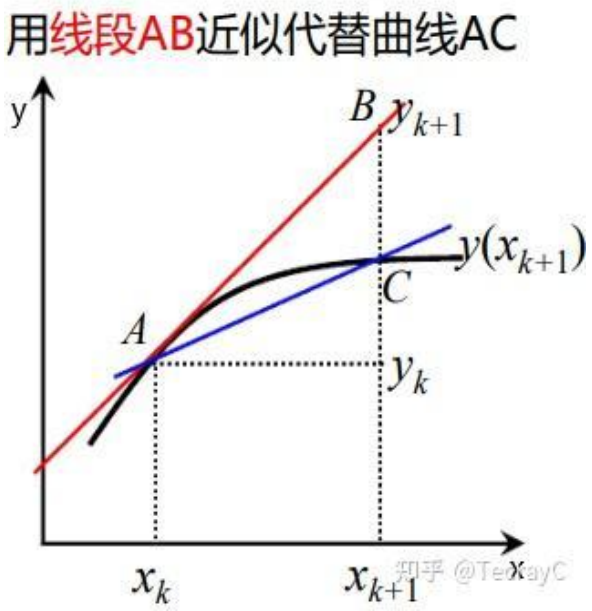
替代若干个block

$$\mathbf{h}_{t+1} = \mathbf{h}_t + f(\mathbf{h}_t, \theta_t)$$

显式欧拉法

$$h(t+\Delta t) = h(t) + \Delta t \cdot \frac{dh(t)}{dt} + \mathcal{O}(\Delta t^2)$$

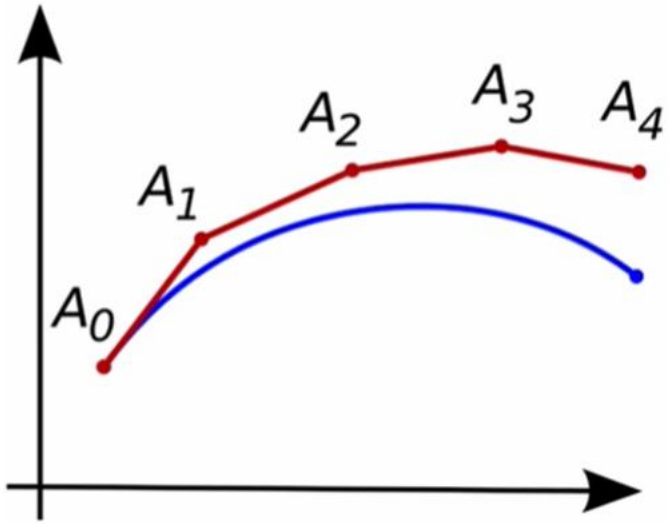
$$h(t+\Delta t) \approx h(t) + \Delta t \cdot f(h(t), t, \theta)$$



残差方法

令$\Delta t = 1$

$$h_{t+1} = h_t + 1 \cdot f(h_t, \theta_t)$$



$$h(t+\Delta t) \approx h(t) + \Delta t \cdot f(h(t), t, \theta)$$

令$\Delta t \to 0$

ode方法

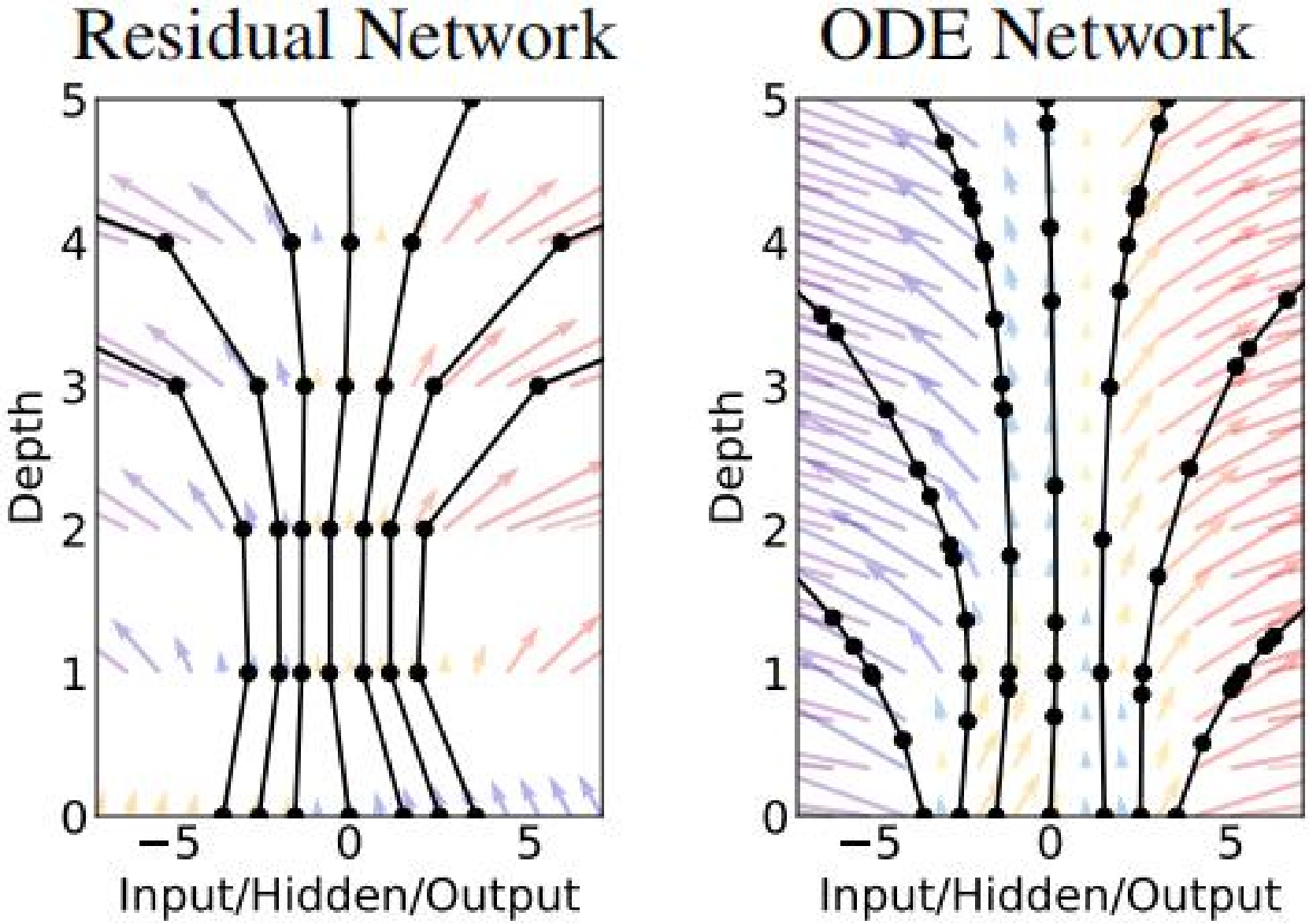
$$\frac{d\mathbf{h}(t)}{dt} = f(\mathbf{h}(t), t, \theta)$$

现代高阶ode solver可以很好的求解该微分方程数值解

$$h(T) = h(0) + \int_0^T f(h(t), t, \theta) dt$$

这里T可以取任何值，从0到T的过程是连续的

注意：T不仅可以取正值，也可以取负值


Residual Network
ODE Network
Depth
Input/Hidden/Output
0
1
2
3
4
5
−5
0
5

它是如何“自行判断”合适的步长 Δt 的?

这句话背后的底层工程机理叫做**自适应步长控制（Adaptive Step Size Control）**。

现代的高阶 ODE 求解器（例如经典的 RK45 算法）绝对不会像 ResNet 那样使用固定的 $\Delta t = 1$。它的底层逻辑是：

1. **双重计算**：在当前时间 t，求解器会尝试迈出一步 Δt。它会同时计算出两个精度的结果。比如一个 4 阶精度的状态近似值 h_4 和一个 5 阶精度的近似值 h_5。
2. **误差估计**：计算这两个近似值的差 $\epsilon_{error} = \|h_5 - h_4\|$。这个差值被用作当前步**局部截断误差**的估计。
3. **动态决策**：
    - 如果 ϵ_{error} 大于你设定的容差（Tolerance），说明在当前复杂的连续向量场中，步子迈得太大了，线性近似失效。求解器会**丢弃这一步，强行缩小** Δt，重新计算。
    - 如果 ϵ_{error} 远小于容差，说明在这个区域状态变化很平缓。求解器不仅会接受这一步，还会在下一步**放大** Δt，以节省计算成本。

主要内容

1.向前计算过程

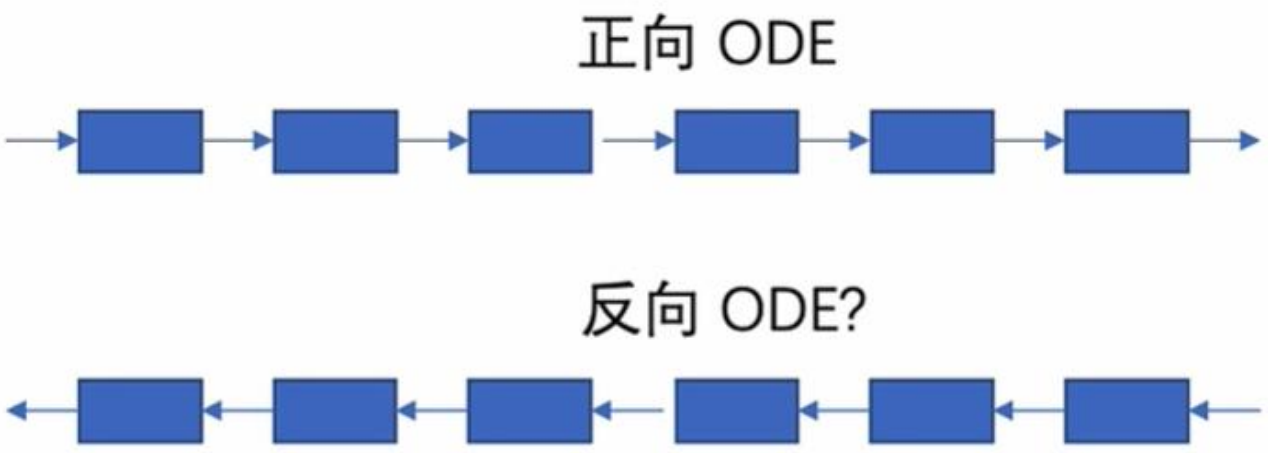
2.向后计算过程

3.优势与应用

矛盾出现了！

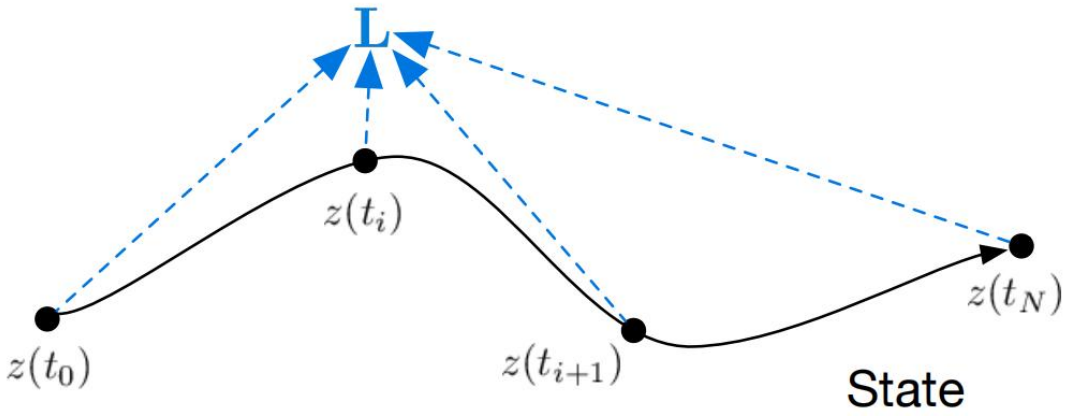
传统深度学习需要把计算图的所有中间激活值全部存在内存里。但这里我们使用ode solver，向前传播可能会调用成千上万次函数f，如果保存所有中间状态，内存复杂度将是 O（L），直接导致内存爆炸 。

伴随敏感度方法
（Adjoint Sensitivity Method）



Neural ODE的损失函数

$$L(\mathbf{z}(t_1)) = L\left(\mathbf{z}(t_0) + \int_{t_0}^{t_1} f(\mathbf{z}(t), t, \theta)dt\right) = L\left(\text{ODESolve}(\mathbf{z}(t_0), f, t_0, t_1, \theta)\right)$$



伴随敏感度方法
（Adjoint Sensitivity Method）

Neural ODE的损失函数

$$L(\mathbf{z}(t_1)) = L\left(\mathbf{z}(t_0) + \int_{t_0}^{t_1} f(\mathbf{z}(t), t, \theta)dt\right) = L\left(\text{ODESolve}(\mathbf{z}(t_0), f, t_0, t_1, \theta)\right)$$

我们现在面对的是一个受约束的非线性优化问题，
我们想要得到损失函数L关于theta的梯度

目标：$\frac{dL}{d\theta}$

伴随敏感度方法

定义伴随敏感度： $a(t) = \frac{\partial L}{\partial z(t)}$

任务一：证明： $$\frac{d\mathbf{a}(t)}{dt} = -\mathbf{a}(t)^{\mathsf{T}} \frac{\partial f(\mathbf{z}(t), t, \theta)}{\partial \mathbf{z}}$$

任务二：证明： $$\frac{dL}{d\theta} = \int_{t_1}^{t_0} \mathbf{a}(t)^{\mathsf{T}} \frac{\partial f(\mathbf{z}(t), t, \theta)}{\partial \theta} dt$$

后涉及硬核数学推导

任务一

设定极小时间步长 $\epsilon \to 0$，状态正向演化近似为：

$$z(t+\epsilon) = z(t) + \epsilon \cdot f(z(t), t, \theta)$$

$$\frac{\partial z(t+\epsilon)}{\partial z(t)} = I + \epsilon \frac{\partial f}{\partial z}$$

根据多元微积分，将 t 时刻的梯度通过 $t+\epsilon$ 时刻的梯度反向表达：

$$a(t) = \frac{\partial L}{\partial z(t+\epsilon)} \frac{\partial z(t+\epsilon)}{\partial z(t)} = a(t+\epsilon)\left(I + \epsilon \frac{\partial f}{\partial z}\right)$$

$$a(t+\epsilon) - a(t) = -\epsilon \cdot a(t+\epsilon) \frac{\partial f}{\partial z}$$

$$\frac{da(t)}{dt} = -a(t)^T \frac{\partial f(z(t), t, \theta)}{\partial z}$$

任务二

演化过程必须严格服从常微分方程的物理约束：$\dot{z}(t) - f(z(t), t, \theta) = 0$

构造**连续时间拉格朗日泛函**：

$$\mathcal{L} = L(z(t_1)) - \int_{t_0}^{t_1} a(t)^T \left(\frac{dz(t)}{dt} - f(z(t), t, \theta) \right) dt$$

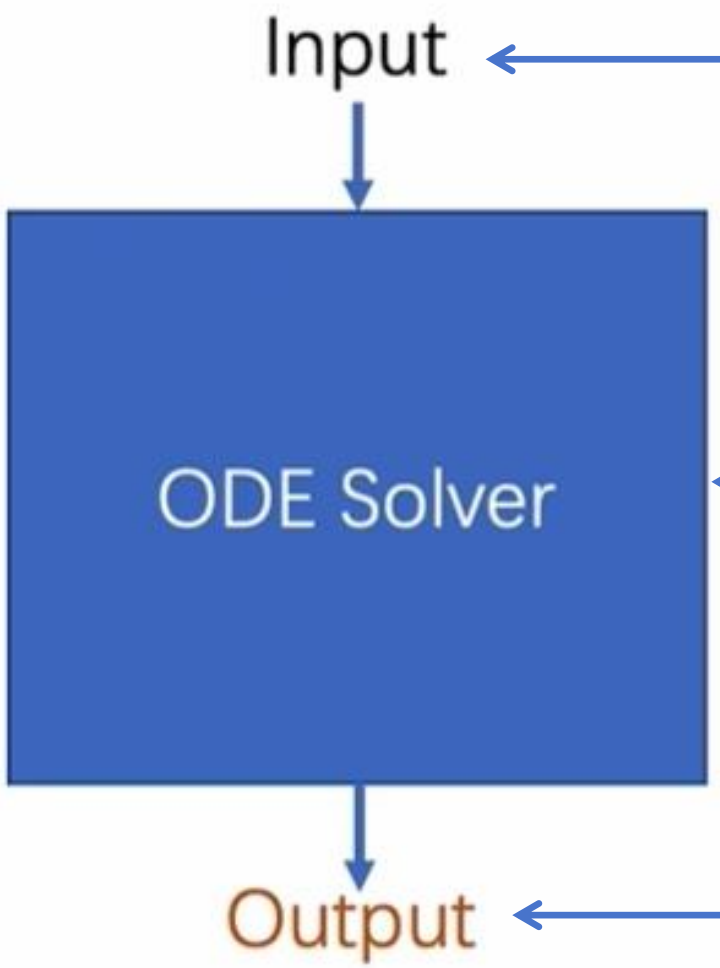
对参数 θ 求全导数：

$$\frac{d\mathcal{L}}{d\theta} = \frac{\partial L}{\partial z(t_1)} \frac{dz(t_1)}{d\theta} - \int_{t_0}^{t_1} a(t)^T \left(\frac{d}{dt} \frac{dz(t)}{d\theta} - \frac{\partial f}{\partial z} \frac{dz(t)}{d\theta} - \frac{\partial f}{\partial \theta} \right) dt$$

化简上式得：

$$\frac{dL}{d\theta} = \int_{t_0}^{t_1} a(t)^T \frac{\partial f}{\partial \theta} dt$$

ODE解的反向模式自动微分

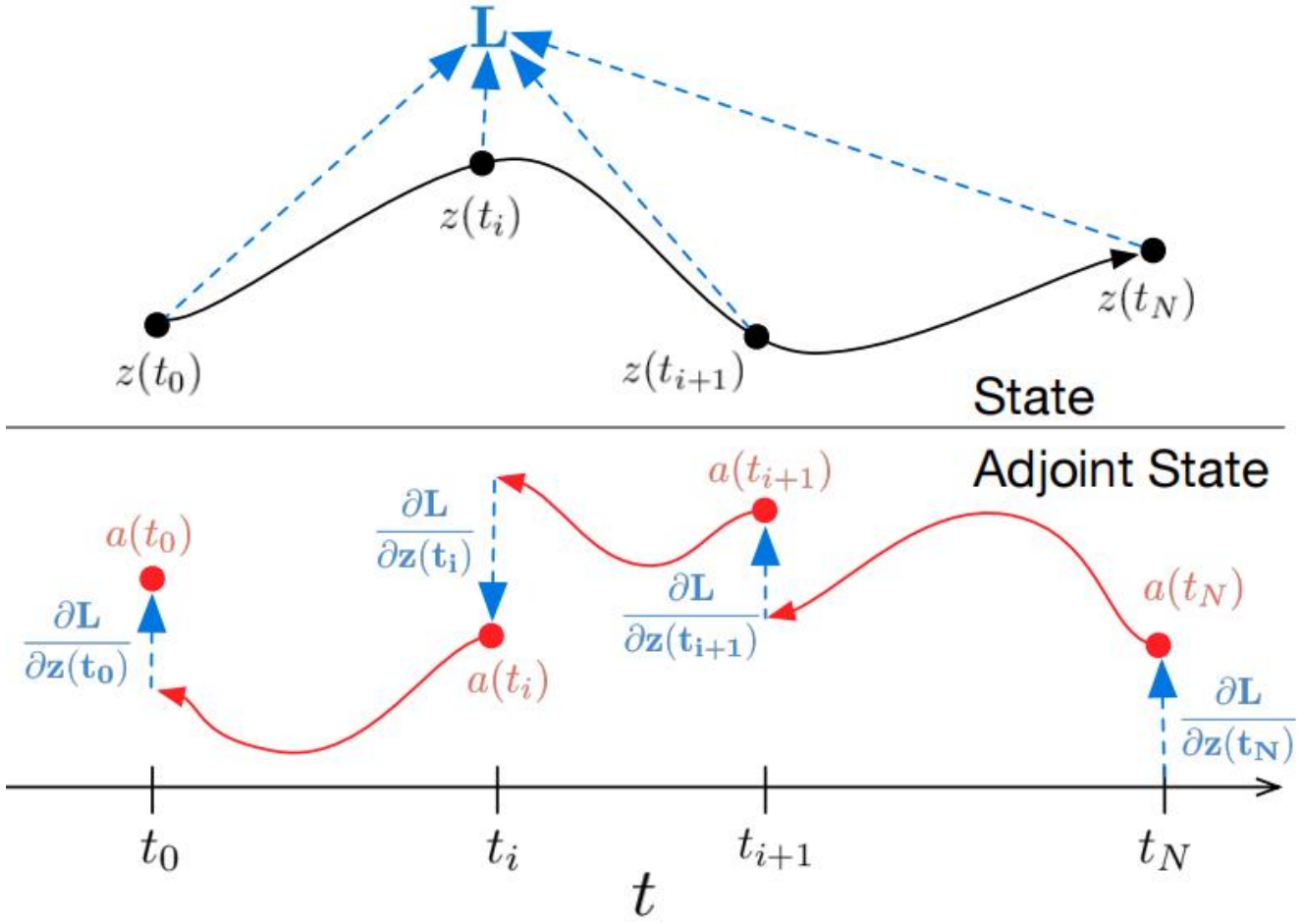


$\mathbf{a}(t)$ f $\mathbf{z}(t)$ θ $t = t_1$

$$\frac{dL}{d\theta} = \int_{t_1}^{t_0} \mathbf{a}(t)^{\mathsf{T}} \frac{\partial f(\mathbf{z}(t), t, \theta)}{\partial \theta} dt$$

$$\frac{d}{dt}\left(\frac{dL}{d\theta}\right) = a(t)^T \frac{\partial f}{\partial \theta}$$

$\frac{dL}{d\theta}$ $t = t_0$



- **断点处的脉冲突变（Jump Condition）**：当逆向时间刚好"撞"上一个观测点（比如 t_{i+1}）时，红色轨迹发生了**不连续的跳跃。**
- **底层物理直觉**：为什么会跳？因为在 t_{i+1} 时刻，总损失 L 直接包含了 $z(t_{i+1})$ 的贡献。当你逆着时间穿过这个点时，系统对 $z(t_{i+1})$ 的总敏感度（Adjoint State），除了包含从未来（t_N 传回来的）累积梯度外，**突然凭空多出了当前时刻局部损失对其产生的直接梯度**。
- **工程妥协**：黑盒 ODE 求解器**无法处理不连续的函数跳跃。**因此，如果存在中间观测点，整个反向传播的积分过程**必须被强行打断**。你不能用一次 `ODESolve` 跑完全程，必须把整个积分区间切分成一个个无观测点的子区间（如 $[t_{i+1}, t_i]$），分段调用求解器。

主要内容

1.向前计算过程

2.向后计算过程

3.应用与优势

- 1.图像分类任务（MNIST）
- 2.连续标准化流（CNF）
- 3.生成式潜函数时间序列模型

1.图像分类任务（MNIST）

在真实的图像分类任务（MNIST）中，验证 Neural ODE 架构在参数量、显存开销和计算动态性上的理论优势。

	Test Error	# Params	Memory	Time
1-Layer MLP†	1.60%	0.24 M	-	-
ResNet	0.41%	0.60 M	$\mathcal{O}(L)$	$\mathcal{O}(L)$
RK-Net	0.47%	0.22 M	$\mathcal{O}(\tilde{L})$	$\mathcal{O}(\tilde{L})$
ODE-Net	0.42%	0.22 M	$\boldsymbol{\mathcal{O}(1)}$	$\mathcal{O}(\tilde{L})$

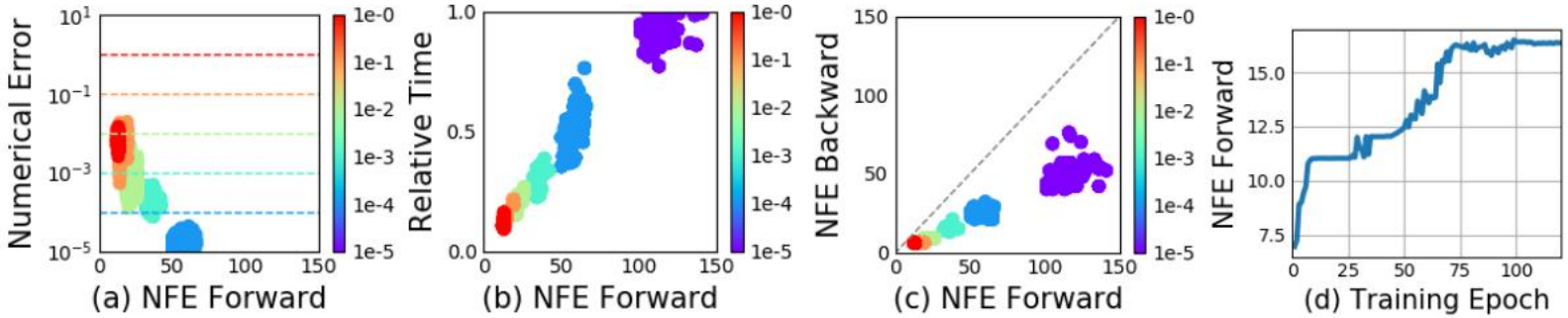


Figure 3: Statistics of a trained ODE-Net. (NFE = number of function evaluations.)

NFE 指在一次前向传播（或反向传播）中，动力学函数 f 总共被调用了多少次 。

2.连续标准化流（CNF）

$$\log p(\mathbf{z}_1) = \log p(\mathbf{z}_0) - \log \left| \det \frac{\partial f}{\partial \mathbf{z}_0} \right|$$

计算该行列式的时间复杂度为O(D^3)

瞬时变量代换定理

$$\frac{\partial \log p(\mathbf{z}(t))}{\partial t} = -\mathrm{tr}\left(\frac{df}{d\mathbf{z}(t)}\right)$$

底层逻辑：将离散的层级映射 $z_{i+1} = f(z_i)$，彻底重塑为连续时间下受速度场驱动的常微分方程演化 $\frac{dz(t)}{dt} = f(z(t), t, \theta)$。

- **极限逼近下的矩阵展开**：在极小时间步 $\Delta t \to 0$ 下，通过前向欧拉离散化，其雅可比矩阵为 $J = I + \Delta t \frac{\partial f}{\partial z}$。
- **特征值连乘与迹的涌现**：利用矩阵分析中 $\det(J) = \prod(1 + \Delta t \cdot \lambda_i)$ 的性质进行多项式展开，抛弃高阶无穷小 $\mathcal{O}(\Delta t^2)$ 后，行列式与迹（Trace）建立了绝对等价：$\det\left(I + \Delta t \frac{\partial f}{\partial z}\right) = 1 + \Delta t \cdot tr\left(\frac{\partial f}{\partial z}\right) + \mathcal{O}(\Delta t^2)$
- **对数概率的连续坍缩**：取对数极限后，原本复杂的行列式对数运算，极其优美地坍缩为线性的迹运算：$\frac{\partial \log p(z(t))}{\partial t} = -tr\left(\frac{df}{dz(t)}\right)$ **【降维打击】**：计算复杂度瞬间从 $\mathcal{O}(D^3)$ 暴降至 $\mathcal{O}(D)$。

3.生成式潜函数时间序列模型

$$\mathbf{z}_{t_0} \sim p(\mathbf{z}_{t_0})$$
$$\mathbf{z}_{t_1}, \mathbf{z}_{t_2}, \ldots, \mathbf{z}_{t_N} = \mathrm{ODESolve}(\mathbf{z}_{t_0}, f, \theta_f, t_0, \ldots, t_N)$$
$$\text{each} \quad \mathbf{x}_{t_i} \sim p(\mathbf{x}|\mathbf{z}_{t_i}, \theta_{\mathbf{x}})$$

1. 公式 (11)：确定潜在初始状态

$\mathbf{z}_{t_0} \sim p(\mathbf{z}_{t_0})$

这表示对于每一条时间序列，首先从一个概率分布中采样出一个初始的潜在状态 $\mathbf{z}_{t_0}$ 。这个状态是“局部”的，它决定了这条特定时间序列的独有特征和起点 。

2. 公式 (12)：生成连续的潜在轨迹

$\mathbf{z}_{t_1}, \mathbf{z}_{t_2}, \ldots, \mathbf{z}_{t_N} = \mathrm{ODESolve}(\mathbf{z}_{t_0}, f, \theta_f, t_0, \ldots, t_N)$

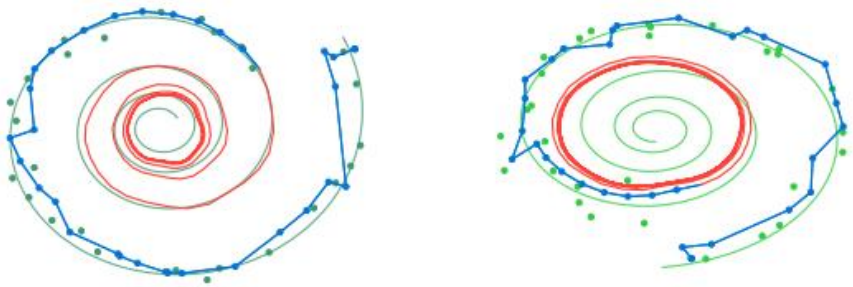
这是模型区别于传统 RNN 的关键所在 。 一旦确立了初始状态 $\mathbf{z}_{t_0}$，后续所有时刻的潜在状态就不再是分步计算的，而是通过一个常微分方程求解器 (ODESolve) 沿着时间积分直接计算出来的 。

- f 和 θ_f 代表了一个神经网络，它定义了状态随时间变化的导数（即动力学规律） 。这个规律是“全局”共享的，意味着所有不同的时间序列都遵循这一套相同的底层演化法则 。
- 因为这是一个连续函数，所以给定的时间点 $t_0, \ldots, t_N$ 可以是任意的、不规则间隔的 。

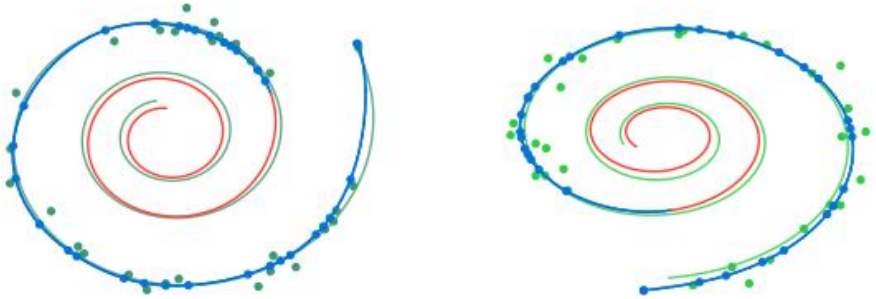
3. 公式 (13)：解码生成观测数据

$\text{each} \quad \mathbf{x}_{t_i} \sim p(\mathbf{x}|\mathbf{z}_{t_i}, \theta_{\mathbf{x}})$

最后，在由 ODE 求解器得出的每个时间点对应的潜在状态 $\mathbf{z}_{t_i}$ 的基础上，通过另一个分布（通常由参数为 $\theta_{\mathbf{x}}$ 的解码器网络构建）将其映射回真实的数据空间，生成最终我们能看到的观测值 $\mathbf{x}_{t_i}$ 。



(a) Recurrent Neural Network



(b) Latent Neural Ordinary Differential Equation

浅绿色实线：真实的螺旋线轨迹 。

绿色散点：加入噪声且不规则采样的观测数据点 。

蓝色实线：模型根据观测点给出的预测与重建轨迹 。

红色实线：模型在观测点结束之后，进行的未来轨迹外推 。

局限性

1. 批量处理的复杂性
   将多个样本拼接成联合ODE进行求解时，为控制误差，可能会导致系统评估次数增加 。

2. 必须满足解的唯一性条件
   为了保证连续动力学具有唯一的解，微分方程必须满足 Picard 存在定理。

3. 需要手动设定误差容忍度
   用户必须手动为前向和反向传播设置合理的误差容差参数。

4. 轨迹反向重建存在数值误差风险
   通过时间逆向求解来重建前向轨迹时，容易偏离原始轨迹并引入额外的数值误差 。